

AcadTrack — System Architecture

Developer: MJR Vertex — Mark Frizas, Rutz Cabrera, Jean Rose Banay

Presented at: Innovex 2026, Indonesia

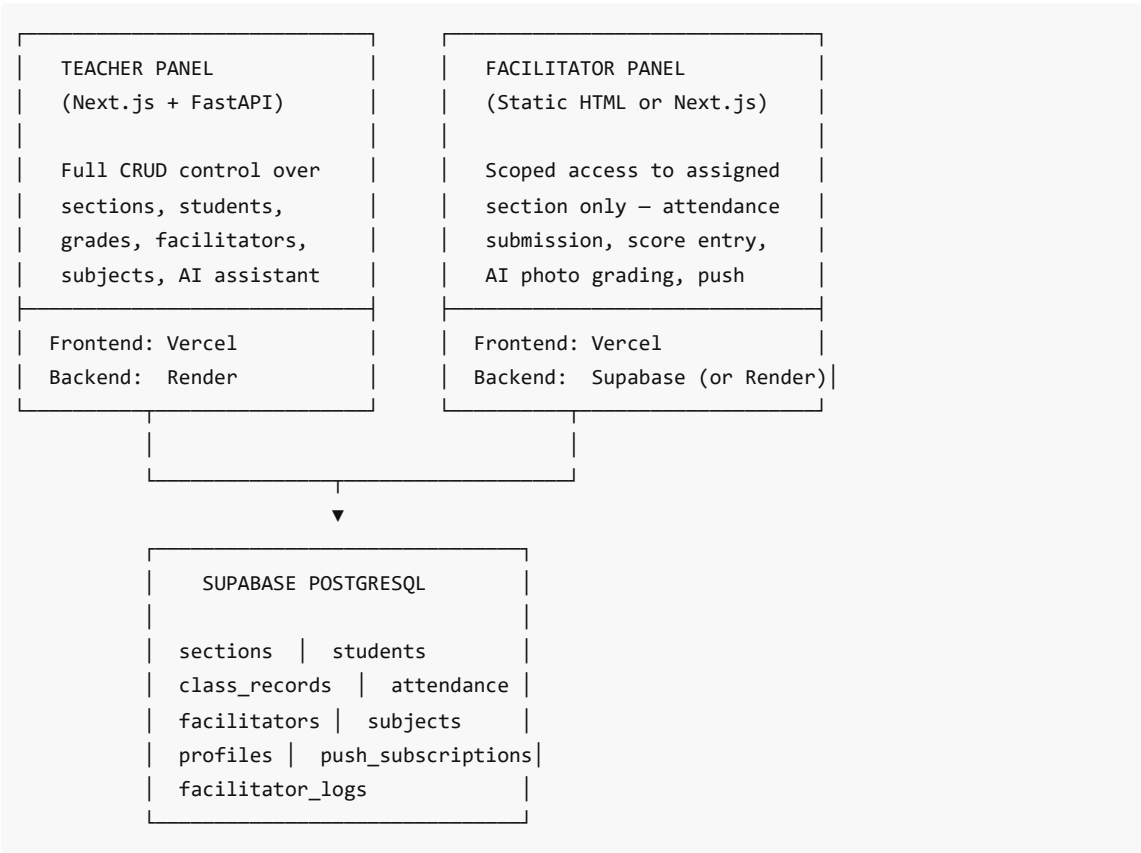
Live: acadtrack.asia (Teacher) | faci-panel.vercel.app (Facilitator)

Table of Contents

- 1. [System Overview](#)
- 2. [How the Teacher Panel Works](#)
- 3. [How the Facilitator Panel Works](#)
- 4. [How the Two Panels Communicate](#)
- 5. [Data Flow for Key Features](#)
- 6. [Authentication Flow](#)
- 7. [Deployment Architecture](#)
- 8. [Full Tech Stack Reference](#)

1. System Overview

AcadTrack is an **academic management system** composed of two independent web applications (Teacher Panel and Facilitator Panel) that share one database.



Why two panels instead of one?

Teachers need **full administrative control** — create sections, add students, set grade weights, manage facilitators, access all data. Facilitators (co-teachers, assistants, substitutes) need **scoped access** — only their assigned section, no ability to change settings or see other classes. Separating them into two panels enforces this boundary naturally while sharing the same database for real-time consistency.

2. How the Teacher Panel Works

2.1 Frontend Architecture

Stack: React 18 + Next.js 14 (App Router) + TypeScript + Tailwind CSS

The teacher panel is a **Next.js single-page application** deployed on Vercel. All pages are client components (`"use client"`) that call the FastAPI backend.

Page Structure:

<code>/</code>	→ Redirects to <code>/login</code>
<code>/login</code>	→ Email/password + Google OAuth
<code>/dashboard</code>	→ Overview with stats, charts, CRUD for schedules/notices/notes
<code>/section</code>	→ Section picker (list of all sections with search)
<code>/section/[id]</code>	→ Section detail – student roster (add/edit/delete students)
<code>/class-record</code>	→ Class Record picker (reuses <code>SectionPickerList</code>)
<code>/class-record/[id]</code>	→ Score grid – 25 modules + 10 activities + AT/PT/QE
<code>/attendance</code>	→ Attendance picker (reuses <code>SectionPickerList</code>)
<code>/attendance/[id]</code>	→ Attendance grid – month view, mark P/A/L per cell
<code>/performance</code>	→ Performance picker (reuses <code>SectionPickerList</code>)
<code>/performance/[id]</code>	→ Charts + ranking table
<code>/facilitators</code>	→ CRUD for facilitators (bcrypt passwords)
<code>/grading-system</code>	→ Subject grade weights (WW/PT/QE/AT percentages)
<code>/about</code>	→ About the system (static)
<code>/help</code>	→ FAQ (static)

API Client (`lib/api.ts`):

Every page imports functions from `lib/api.ts` , a thin wrapper around `fetch` :

```
// Example: fetching sections
const sections = await apiGet("/api/sections");

// Example: saving attendance
await apiPost(`/api/sections/${sectionId}/attendance`, {
  date: "2026-01-15",
  records: [{ student_id: "...", status: "Present" }]
});
```

The API client automatically:

1. Reads the Supabase access token from the current session
2. Attaches it as `Authorization: Bearer <token>`
3. Sets JSON content-type headers
4. Redirects to `/login` if the server returns 401

Data Caching:

A `useCachedData` hook caches API responses in `localStorage` with a 5-minute TTL (stale-while-revalidate). Cache keys follow `dash_cache_*` for dashboard data and `list_cache_*` for section lists.

Real-Time Updates (Supabase Realtime):

The frontend subscribes to PostgreSQL changes using Supabase Realtime channels:

Component	Watches	Effect
SectionPickerList	sections, students	Refreshes list when data changes
Dashboard	sections, students, class_records, attendance	Updates stat cards
AttendanceGrid	attendance	Auto-updates cells (2s debounce skips own edits)
ClassRecordGrid	class_records	Reloads scores (skips own edits)
PerformanceDetail	students, class_records	Refreshes charts
FacilitatorsPage	facilitators	Refreshes table
GradingSystem	subjects	Updates weight configs

This means **when a facilitator submits scores, the teacher sees them instantly** without refreshing.

2.2 Backend Architecture

Stack: FastAPI + Python 3.12 + Uvicorn + SQLAlchemy 2 (async) + asyncpg

The backend is a **REST API** deployed on Render. It handles all data operations, verifies auth, and enforces ownership.

Router Structure (`backend/app/routers/`):

Router	Prefix	Purpose
<code>dashboard.py</code>	<code>/api</code>	Profile, schedules, notices, notes, bulk dashboard data
<code>sections.py</code>	<code>/api</code>	Sections CRUD, students CRUD, bulk student import
<code>records.py</code>	<code>/api/sections</code>	Attendance (delete+insert per date), class records (upsert)
<code>grading.py</code>	<code>/api/subjects</code>	Subject grade weights CRUD
<code>facilitators.py</code>	<code>/api/facilitators</code>	Facilitator CRUD with bcrypt passwords
<code>ai.py</code>	<code>/api</code>	AI assistant — Groq primary, Gemini fallback
<code>push.py</code>	<code>/api/push</code>	VAPID public key, push subscription management

Auth Enforcement:

Every endpoint uses a dependency `get_current_teacher` that:

- 1. Extracts the `Authorization: Bearer <token>` header

2. Fetches Supabase's JWKS public key and verifies the token signature (ES256)
3. Extracts the teacher's UUID (`sub` claim)
4. Loads the teacher's profile from the database
5. Returns a `CurrentTeacher` object

Then an `own_section()` dependency checks that the requested section belongs to that teacher (returns 404/403 if not). This prevents any teacher from accessing another teacher's data.

Database Models (1:1 with existing schema):

Model	Table	Key Fields
Profile	<code>profiles</code>	<code>id</code> , <code>full_name</code> , <code>email</code> , <code>avatar_url</code> , <code>theme</code>
Section	<code>sections</code>	<code>id</code> , <code>teacher_id</code> , <code>title</code> , <code>subject</code> , <code>room</code> , <code>semester</code> , <code>school_year</code> , <code>quarter</code>
Student	<code>students</code>	<code>id</code> , <code>section_id</code> , <code>full_name</code> , <code>gender</code>
Attendance	<code>attendance</code>	<code>id</code> , <code>date</code> , <code>section</code> , <code>student_name</code> , <code>status</code> (P/A/L), <code>quarter</code> , <code>teacher_id</code>
ClassRecord	<code>class_records</code>	<code>id</code> , <code>student_id</code> , <code>section_id</code> , <code>quarter</code> , <code>module_1..25</code> , <code>activity_1..10</code> , <code>at_pt_1</code> , <code>pt_2</code> , <code>qe</code>
Facilitator	<code>facilitators</code>	<code>id</code> , <code>full_name</code> , <code>section</code> , <code>subject</code> , <code>account_id</code> , <code>teacher_id</code> , <code>password</code> (bcrypt)
Subject	<code>subjects</code>	<code>id</code> , <code>teacher_id</code> , <code>name</code> , <code>ww_percent</code> , <code>pt_percent</code> , <code>exam_percent</code> , <code>attendance_percent</code> , <code>passing_grade</code>
PushSubscription	<code>push_subscriptions</code>	<code>id</code> , <code>user_type</code> , <code>user_id</code> , <code>endpoint</code> , <code>subscription</code> (JSONB)
FacilitatorLog	<code>facilitator_logs</code>	<code>id</code> , <code>facilitator_id</code> , <code>time_in</code> , <code>time_out</code>

2.3 Grade Computation

Grade weights are **not hardcoded**. Each teacher configures per-subject percentages for Written Work, Performance Tasks, Quarterly Exam, and Attendance. The frontend computes grades using `lib/grading.ts` :

```

WW% = (student's average WW score) × (subject.ww_percent / 100)
PT% = (student's average PT score) × (subject.pt_percent / 100)
QE% = (student's QE score) × (subject.exam_percent / 100)
AT% = (attendance rate) × (subject.attendance_percent / 100)

Final Grade = WW% + PT% + QE% + AT%
```

Both panels use the **identical grading engine** (`grading.ts` in teacher-panel, `web/lib/grading.ts` in faci-panel), so they always compute the same final grade.

3. How the Facilitator Panel Works

3.1 Legacy Static App (Currently Live)

Stack: HTML5 + CSS3 + Vanilla JavaScript + Supabase JS Client

The legacy app is a **multi-page static site** — each page is a standalone HTML file with inline JavaScript that connects directly to Supabase using the anon key.

Pages:

Page	File	Purpose
Login	login.html	Authenticate with account ID + password
Dashboard	index.html	Overview cards, class ranking, bottom drawers
Attendance	attendance.html	Mark P/A/L per student for a selected date
Class Records	record.html	Score grid + AI photo-to-score
Profile	profile.html	Avatar upload, co-facilitators, logout

Key JavaScript modules (js/):

File	Purpose
grading.js	Grade computation engine (shared logic with teacher panel)
faci-session.js	Session heartbeat — opens <code>facilitator_logs</code> row on load, stamps <code>time_out</code> on unload, pings every 30s
mjr-notify.js	Push notification setup — registers service worker, subscribes to VAPID push, shows toasts
mjr-guard.js	Client protection — blocks right-click, F12, DevTools shortcuts, masks URLs
mjr-sw.js	Service worker — receives push events, shows OS notifications

Auth Flow (legacy):

1. User enters account ID + password on `login.html`
2. **Fast path:** browser queries Supabase `facilitators` table directly with `account_id` , then verifies bcrypt hash in-browser
3. **Fallback path:** if bcrypt library unavailable, POSTs to FastAPI `/api/faci/login`
4. On success, stores `faci_id` , `faci_name` , `faci_section` , `faci_subject` , `faci_teacher_id` in `localStorage`
5. Redirects to `index.html`
6. Every non-login page checks `localStorage` for auth — missing keys redirect to login

3.2 Next.js Migration (In Progress)

Stack: React 18 + Next.js 14 (App Router) + TypeScript

The Next.js migration is a **faithful port** of the static app with one key difference: instead of talking to Supabase directly from the browser, it calls the FastAPI backend (same as the teacher panel).

Pages: `/login` , `/` , `/attendance` , `/record` , `/profile` — identical UI/UX to the static pages, same CSS ported verbatim.

Key differences from static app:

- API calls go to FastAPI (JWT-authenticated) instead of Supabase JS client

- No Supabase Realtime live-refresh toasts (since browser no longer holds a Supabase client)
- React hooks replace DOM manipulation
- Push notifications register through FastAPI instead of direct Supabase upsert

3.3 Backend

Stack: FastAPI + Python 3.12 + Uvicorn + SQLAlchemy 2 (async) + asyncpg

The faci-panel backend is a **separate FastAPI app** that connects to the **same Supabase PostgreSQL** database. It provides JWT-based auth for facilitators.

Auth flow (backend):

1. POST `/api/faci/login` — accepts `account_id` + `password`, verifies bcrypt hash, returns JWT (HS256, 30-day expiry)
2. GET `/api/faci/me` — resolves JWT, returns faci profile (also used as "account still exists?" check)
3. POST `/api/faci/heartbeat` — keeps status Active, updates `last_login`

Data endpoints all scope to the facilitator's own section + teacher server-side:

- GET `/api/faci/section` — resolved assigned section
- GET `/api/faci/students` — students in the section
- GET `/api/faci/class-records` — class records for the section
- POST `/api/faci/class-records` — bulk upsert scores
- GET `/api/faci/attendance?date=X` — attendance rows
- POST `/api/faci/attendance` — replace a day's attendance
- GET `/api/faci/subjects` — teacher's grade weights
- POST `/api/vision-analyze` — AI photo grading (Gemini + Groq)

4. How the Two Panels Communicate

4.1 Shared Database (Primary Mechanism)

Both panels read from and write to the **exact same PostgreSQL tables**. There is no API-to-API communication — they share state through the database.

```
Teacher writes a score      → class_records table updated
Facilitator reads scores    → reads from same class_records table → sees teacher's change
Facilitator marks attendance → attendance table updated
Teacher views attendance    → reads from same attendance table → sees facilitator's change
```

This means **every change is instantly visible** to the other panel on the next data fetch. No synchronization or duplicate data entry needed.

4.2 Supabase Realtime (Legacy Static FACI Only)

The static faci-panel pages subscribe to PostgreSQL changes using Supabase's JS Realtime client:

- **Attendance page:** listens on `public:attendance` channel. When the teacher modifies attendance for the same section+date, a toast appears and the list auto-reloads.
- **Class Records page:** listens on `public:class_records` channel. When the teacher changes a score, a toast appears and the table auto-reloads.

The teacher panel also uses Realtime (for both teacher and faci changes), creating a **bidirectional live update** loop:

```
Teacher edits score
→ PostgreSQL row changes
→ Supabase Realtime triggers
→ Faci panel auto-reloads (shows toast "Teacher updated scores")
```

```
Facilitator marks attendance
→ PostgreSQL row changes
→ Supabase Realtime triggers
→ Teacher panel auto-reloads (updates dashboard stats)
```

4.3 Push Notifications (Web Push / VAPID)

Push notifications add an **offline real-time layer** — alerts arrive even when the app tab is closed.

Flow:

```
Teacher edits attendance/score
→ Supabase Database Webhook (configured in Supabase Dashboard)
→ Triggers Vercel serverless function /api/push-notify (in teacher-panel repo)
→ Looks up push_subscriptions for affected section's facilitator
→ Sends VAPID Web Push via web-push library
→ Facilitator's service worker (mjr-sw.js) receives push event
→ OS-level notification displayed
```

Facilitator subscription: The facilitator's browser registers with `mjr-sw.js`, creates a `PushSubscription`, and stores it in `push_subscriptions` with `user_type='faci'`.

4.4 Heartbeat (Keep-Alive)

Because both backends run on **Render's free tier** (which spins down after 15 minutes of inactivity), both panels implement heartbeat mechanisms:

- **Browser:** every page sends a 1-second ping to `/api/ping` to keep the backend warm while the user is active
- **GitHub Actions:** a cron job pings the backend every 5 minutes outside of browser hours
- **UptimeRobot:** monitors the backend URL every 5 minutes as a third layer

5. Data Flow for Key Features

5.1 Creating a Facilitator

```
Teacher Panel
→ Teacher fills form: name, section, subject, account_id, password
→ Frontend POSTs to /api/facilitators
→ Backend hashes password with bcrypt
→ Backend inserts row into facilitators table
→ Supabase Realtime notifies faci-panel (if subscribed)
→ Facilitator can now log in at faci-panel.vercel.app
```

5.2 Attendance Marking

Teacher marks attendance:

- Teacher panel frontend POSTs to `/api/sections/{id}/attendance`
- Backend deletes existing rows for that date+section+teacher
- Backend inserts new attendance rows
- Response returns to teacher

Facilitator marks attendance:

- Faci panel frontend POSTs to `/api/faci/attendance` (or Supabase directly in legacy)
- Same delete-insert pattern
- Database row changes → Realtime triggers on teacher panel
- Teacher's attendance grid auto-refreshes

Date lock: once attendance is submitted for a past date, the system locks it.

Facilitators cannot edit locked dates; only the teacher can unlock.

5.3 Score Recording

Teacher records scores:

- Opens class record grid for a section
- Edits contentEditable cells (module scores, activity scores, AT, PT, QE)
- Each change triggers an API call to `/api/sections/{id}/class-records`
- Backend upserts `class_records` (INSERT ON CONFLICT UPDATE)

Facilitator records scores (via AI photo):

- Opens record page, selects "Upload Photo"
- Camera/gallery captures image, downsized to JPEG
- POST to `/api/vision-analyze` with `type="record"`
- Gemini/Groq vision AI extracts scores from the photo
- Review modal shows extracted scores (editable)
- On Apply, POSTs bulk upsert to `/api/faci/class-records`

Both panels share the same data – teacher sees facilitator's scores instantly.

5.4 Performance Analytics

Teacher views analytics:

- Opens `/performance/{sectionId}`
- Frontend fetches all `class_records` + students for the section
- Grading engine computes final grades per student
- `Chart.js` renders line chart of performance trends
- Ranking table shows highest/lowest performers

The analytics are computed entirely on the frontend (no special analytics API) using the same grading engine both panels share.

5.5 AI Assistant

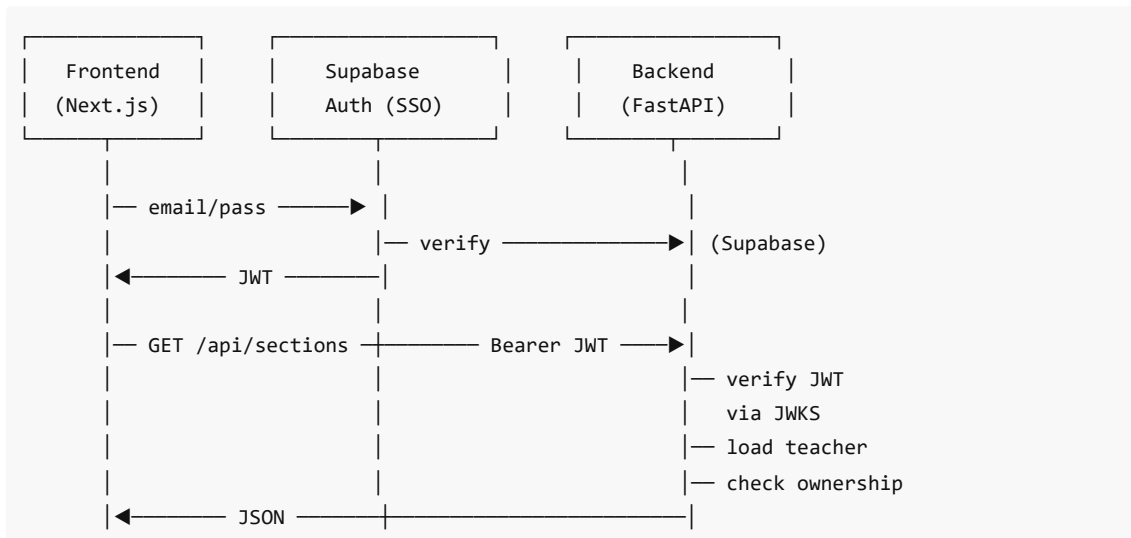
Teacher asks AI a question:

- Types a query in the floating AI chat widget
- Frontend first checks deterministic local lookups (e.g. "highest score")

- If local lookup can't answer → POST to /api/ai-evaluate
- Backend builds context: section info, recent records, student data
- Backend sends to Groq (primary) with the context + question
- If Groq fails → falls back to Gemini
- AI response returned to frontend and displayed in chat widget

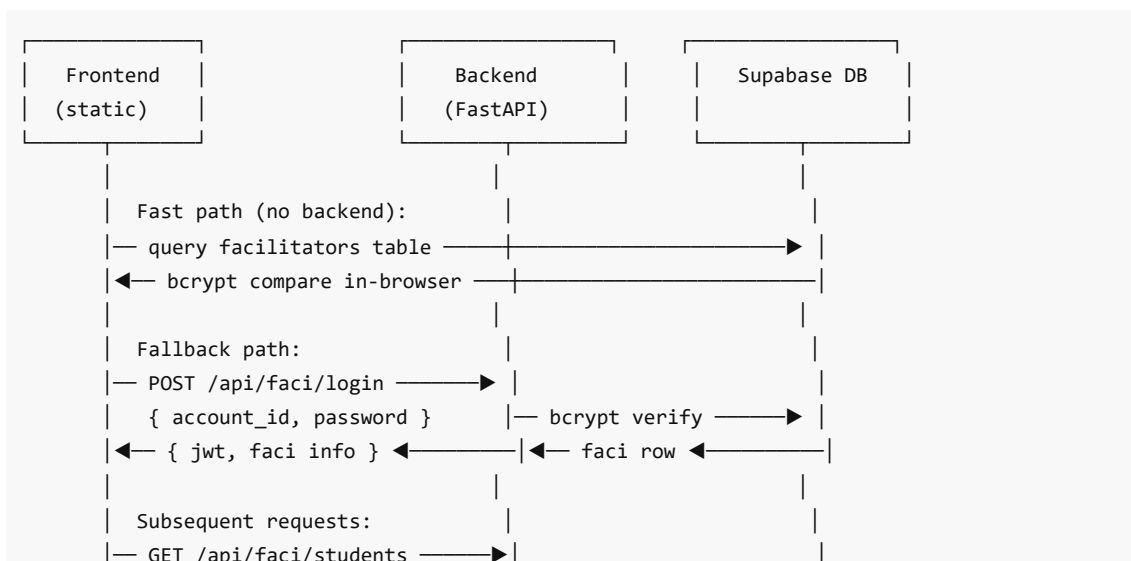
6. Authentication Flow

6.1 Teacher Panel Auth



1. Teacher logs in via Supabase Auth (email/password or Google OAuth)
2. Supabase returns a JWT access token (ES256-signed)
3. Frontend stores the token and attaches it to every API call
4. Backend verifies the token using Supabase's JWKS public key endpoint
5. Backend extracts the teacher's UUID and loads their profile
6. Backend checks section ownership before any data operation

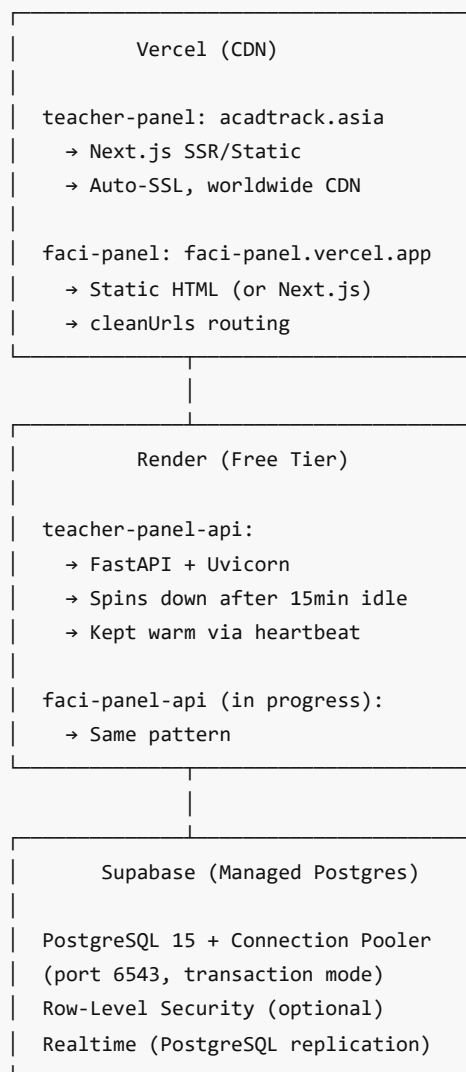
6.2 Facilitator Panel Auth





1. Facilitator enters account ID + password on login page
2. **Fast path:** browser queries Supabase directly (avoids cold-start on Render free tier)
3. **Fallback:** if bcrypt unavailable in-browser, POSTs to FastAPI
4. Backend verifies bcrypt hash, issues JWT (HS256, 30 days)
5. Frontend stores JWT in `localStorage`
6. Every API call includes `Authorization: Bearer <token>`
7. Backend decodes JWT, resolves facilitator, scopes all queries

7. Deployment Architecture



Backend (Render):

- Python 3.12, deployed via `render.yaml` Blueprint
- Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Connection pooler-safe (`NullPool` , `statement_cache_size=0`)

- Uses Supabase Transaction Pooler (port 6543) with `postgresql+asyncpg://` scheme

Frontend (Vercel):

- Teacher panel: Next.js app in `frontend/` directory
- FACI panel: Static HTML in root (or Next.js in `web/` when migration promoted)
- Both auto-deploy from `main` branch

Database (Supabase):

- Shared between both panels — no data migration needed
- Connection pooling via pgBouncer (port 6543)
- Auth managed through Supabase Auth service (not built-in)

8. Full Tech Stack Reference

Layer	Technology	Purpose
Frontend (Teacher)	React 18 + Next.js 14 + TypeScript	UI framework
Frontend (Teacher)	Tailwind CSS	Styling
Frontend (Teacher)	Chart.js	Performance charts
Frontend (Teacher)	Lucide Icons	UI icons
Frontend (FACI)	HTML5 + CSS3 + Vanilla JS	Legacy static app
Frontend (FACI)	Next.js 14 + TypeScript	Migration target
Frontend (FACI)	Font Awesome	UI icons
Backend	FastAPI + Python 3.12	REST API
Backend	Uvicorn	ASGI server
Backend	SQLAlchemy 2 (async)	ORM
Backend	asyncpg	PostgreSQL driver
Backend	PyJWT	JWT auth
Backend	bcrypt	Password hashing
Backend	httpx	Async HTTP client (AI providers)
Backend	pywebpush	Web Push (VAPID)
Database	Supabase PostgreSQL 15	Data storage
Database	Supabase Realtime	Live sync
Database	pgBouncer (pooler)	Connection pooling
AI	Groq (primary)	AI assistant + vision
AI	Gemini (fallback)	AI assistant + vision

Hosting	Vercel	Frontend (auto-SSL, CDN)
Hosting	Render	Backend (free tier)
Hosting	Supabase	Database (managed)
Monitoring	GitHub Actions + UptimeRobot	Keep-alive
Notifications	Web Push API + VAPID	Push notifications